

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL 2-PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25 50
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	D. Snyana Bai	Reg.No.:	192571067
Department:	CS & BIOSCIENCE	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I D. Snyana Bai (192571067) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

D. Snyana Bai
Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1	2	3	2	1	2
Q2	2	2	2	1	2
Q3	1.5	2	1.5	0.5	1.5
Q4	1.5	2.5	1.5	0.5	1.5
Q5	2	2.5	2	1	2

RUBRICS FOR CALCULATION:

43

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20% 2	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30% 3	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20% 2	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10% 1	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20% 2	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. DMA-Based Data Transfer Algorithm

Problem Understanding

Design a DMA-based algorithm that transfers data from an I/O device to memory while minimizing CPU involvement and handling interrupts.

Pseudocode

ALGORITHM DMA_Data_Transfer

BEGIN

 INITIALIZE DMA_Controller

 INITIALIZE IO_Device

 INITIALIZE Memory_Buffer

 CPU_CONFIGURE_DMA(
 Source = IO_Device,
 Destination = Memory_Buffer,
 Transfer_Size = N
)

 START_DMA_TRANSFER

 WHILE DMA_Transfer_Not_Completed DO
 CPU_EXECUTE_OTHER_TASKS
 END WHILE

 IF DMA_Transfer_Completed THEN
 GENERATE_INTERRUPT
 END IF

 ON_INTERRUPT:
 SAVE_CPU_STATE
 VERIFY_TRANSFER_STATUS

 IF Status = SUCCESS THEN
 DISPLAY "Transfer Completed Successfully"
 ELSE
 DISPLAY "Transfer Failed"
 END IF

RESTORE_CPU_STATE

END

Key Features

- CPU only initializes DMA.
- DMA transfers data directly.
- Interrupt generated only after completion.
- Minimal CPU intervention.

Q2. Interrupt Handling System Using Vectored Interrupts

Problem Understanding

Develop an interrupt system that prioritizes vectored interrupts from multiple devices and dispatches the correct ISR.

Pseudocode

ALGORITHM Vectored_Interrupt_Handler

BEGIN

 INITIALIZE Interrupt_Vector_Table

 WHILE TRUE DO

 WAIT_FOR_INTERRUPT

 RECEIVE Interrupt_Request

 DETERMINE Priority_Level

 SELECT Highest_Priority_Interrupt

 FETCH_ISR_ADDRESS_FROM_VECTOR_TABLE

 SAVE_CPU_CONTEXT

 CALL_ISR(ISR_ADDRESS)

 RESTORE_CPU_CONTEXT

END WHILE

END

ISR_DEVICE_A

BEGIN

PROCESS_DEVICE_A_REQUEST

END

ISR_DEVICE_B

BEGIN

PROCESS_DEVICE_B_REQUEST

END

ISR_DEVICE_C

BEGIN

PROCESS_DEVICE_C_REQUEST

END

Key Features

- Supports multiple devices.
- Uses vectored interrupt table.
- Executes highest-priority ISR first.
- Reduces interrupt latency.

Q3. Dynamic Selection Between Memory-Mapped and I/O-Mapped I/O

Problem Understanding

Select the appropriate I/O technique based on latency and bandwidth requirements.

Pseudocode

ALGORITHM Select_IO_Method

INPUT:

Latency_Requirement

Bandwidth_Requirement

BEGIN

IF Latency_Requirement = HIGH OR

Bandwidth_Requirement = HIGH THEN

IO_Mode $\leftarrow$ MEMORY_MAPPED_IO

ELSE

IO_Mode $\leftarrow$ IO_MAPPED_IO

END IF

DISPLAY "Selected Mode = ", IO_Mode

END

Extended Version

IF Latency < 5 microseconds THEN

USE Memory_Mapped_IO

ELSE IF Bandwidth > 1 GBps THEN

USE Memory_Mapped_IO

ELSE

USE IO_Mapped_IO

END IF

Key Features

- Dynamically selects optimal technique.
- Supports performance-based decision making.
- Reduces communication overhead.

Q4. Bus Arbitration Algorithm for PCIe, USB and Thunderbolt

Problem Understanding

Manage communication among multiple devices connected through PCIe, USB and Thunderbolt while ensuring fair access to the bus.

Pseudocode

ALGORITHM Bus_Arbitration

BEGIN

CREATE Device_Queue

ADD PCIe_Devices

ADD USB_Devices
ADD Thunderbolt_Devices

WHILE Device_Queue IS NOT EMPTY DO

 SELECT_NEXT_DEVICE_USING_ROUND_ROBIN

 GRANT_BUS_ACCESS

 TRANSMIT_DATA

 RELEASE_BUS

 MOVE_DEVICE_TO_END_OF_QUEUE

END WHILE

END

Enhanced Priority Handling

IF Device_Type = Thunderbolt THEN

 Priority $\leftarrow$ HIGH

ELSE IF Device_Type = PCIe THEN

 Priority $\leftarrow$ MEDIUM

ELSE

 Priority $\leftarrow$ NORMAL

END IF

Key Features

- Fair bus allocation.
- Prevents starvation.
- Supports multiple interface technologies.
- Efficient resource utilization.

Q5. Hybrid I/O Communication Mechanism

Problem Understanding

Implement a hybrid communication system that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Pseudocode

ALGORITHM Hybrid_IO_System

BEGIN

FOR EACH Device IN Device_List DO

IF Device.Speed = LOW THEN

WHILE Device_Data_Not_Ready DO

POLL_DEVICE

END WHILE

READ_DEVICE_DATA

ELSE

CONFIGURE_DMA(Device)

START_DMA_TRANSFER

WAIT_FOR_INTERRUPT

IF Interrupt_Received THEN

PROCESS_TRANSFER_RESULT

END IF

END IF

END FOR

END

Modular Version

FUNCTION Polling_Mode(Device)

BEGIN

WHILE Device_Not_Ready DO

POLL_DEVICE

END WHILE

READ_DATA

END

FUNCTION DMA_Mode(Device)

BEGIN

 CONFIGURE_DMA

 START_DMA

 WAIT_FOR_INTERRUPT

 PROCESS_DATA

END

Key Features

- Polling used only where appropriate.
- DMA handles high-speed devices.
- Interrupt-driven completion notification.
- Optimized CPU utilization.